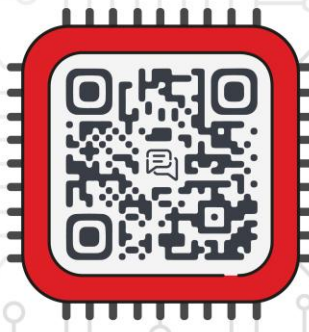




9



8



960 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Dijkstra



د. عملي مشترك

RB Informatics; 6/26/2025

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

بعد أن تعرفنا في المحاضرة السابقة على بنية معطيات جديدة وهي ال Graph وخوارزميات التجوال المطبقة عليها، وأقصر طريق في البيان الغير موزون، سنتعرف في هذه المحاضرة على خوارزميات شهيرة تطبق على البيان الموزون

((Dijkstra))

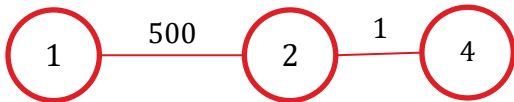
Are you ready! Let's get started

## Shortest path in weighted Graph

### Dijkstra

- تستخدم هذه الخوارزمية لحساب أقصر طريق بين المصدر وجميع العقد في البيان الموزون
- فلحساب مسافة الطريق في البيان الموزون بين مجموعة عقد نقوم بجمع أوزان الوصلات في ذلك الطريق
- فلمعرفة أقصر طريق نأخذ الوصلة صاحبة الوزن الأقل عندما يكون لذي أكثر من خيار ممكن.

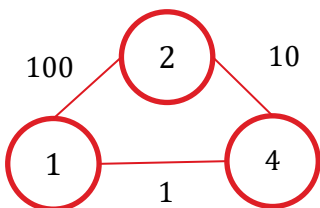
1. مثال: احسب كلفة الطريق من العقدة 1 إلى العقدة 4



الحل:

$$1 + 500 = 501$$

2. مثال: احسب أقصر طريق من العقدة 2 إلى 1:



لدينا طريقين

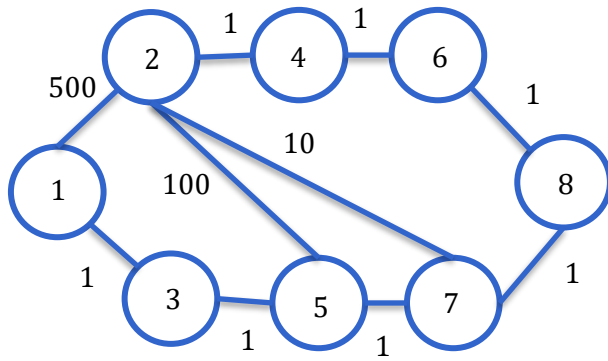
1. إما من 2 ← 1

ستكون التكلفة: 100

2. أو من 2 ← 4 ← 1

ستكون التكلفة: 11 = 10 + 1

ننسلك الطريق الثاني لأنه صاحب التكلفة الأقل



### لنعود إلى خوارزمية Dijkstra:

ليكن لدينا هذا البيان:

نريد أقصر مسافة بين المصدر 1 وباقي العقد  
ما الذي سنحتاجه لحساب أقصر مسافة؟

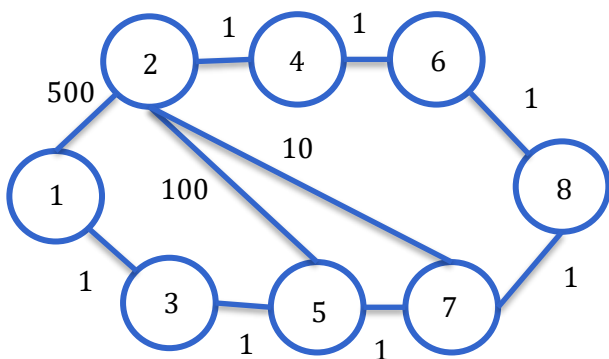
1. في المحاضرة السابقة استخدمنا مصفوفة visited لأننا كنا نسجل أن العقدة مزارة بمجرد أول زيارة لها ولا نعود لها مرة أخرى ولكن هنا قد تكون أول زيارة للعقدة ليست أقصر طريق لها من المصدر، إذا لن نحتاج إلى مصفوفة visited

### ملاحظة

يمكن تمثيل خوارزمية Dijkstra بطريقة أخرى نستخدم فيها مصفوفة visited ولكن لن نعتمدها في مقرر العملي.

2. نستخدم مصفوفة distance لتخزين المسافة بين المصدر وباقي العقد والتحديث عليها

3. سنستخدم queue، لأننا سنأخذ كل عقدة ثم مجاوراتها ولكن هذا ال queue لن نخزن العقدة فقط، بل المسافة أيضاً (وزن الوصلات) أي سيكون عبارة عن pair

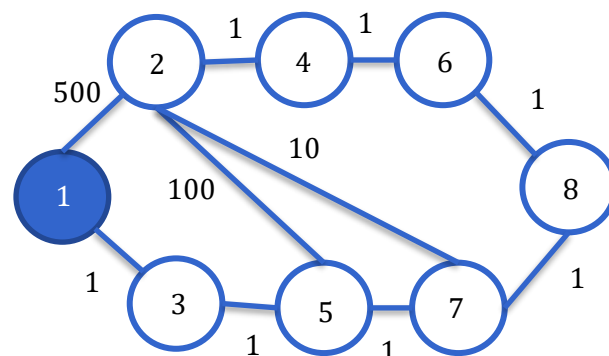


| vertices | 1         | 2         | 3         | 4         | 5         | 6         | 7         | 8         |
|----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| distance | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ |

Queue

The queue is empty -no node added-

■ نبدأ من source والذي هو 1 نجعل المسافة عنده 0 ونضيفه إلى ال queue

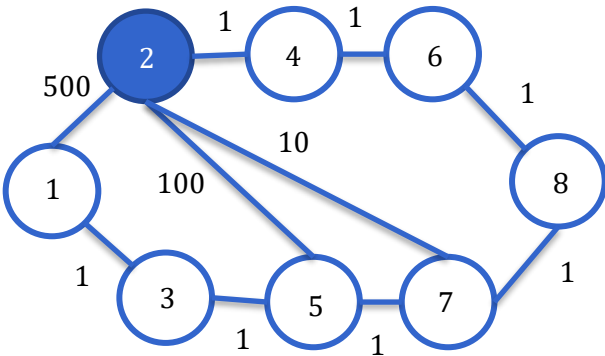


| vertices | 1 | 2         | 3         | 4         | 5         | 6         | 7         | 8         |
|----------|---|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| distance | 0 | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ |

Queue

{1,0}

■ نحذف العنصر في بداية ال queue نقارن وزنه مع أفضل وزن للعقدة نجدهم متساويين فنفحص جيرانهم ونضيفهم إلى queue ونحدث قيمهم في المصفوفة ال distance

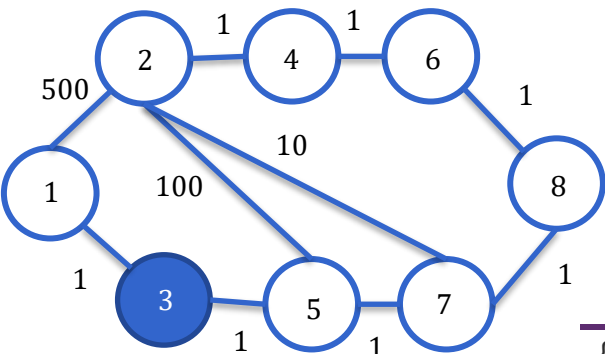


| vertices | 1 | 2   | 3 | 4         | 5         | 6         | 7         | 8         |
|----------|---|-----|---|-----------|-----------|-----------|-----------|-----------|
| distance | 0 | 500 | 1 | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ |

Queue

{2,500} {3,1}

■ نحذف العنصر في بداية queue نقارن وزنه مع أفضل وزن للعقدة نجدهم متساويين فنفحص جيرانهم ونضيفهم إلى queue ونحدث قيمهم في المصفوفة ال distance

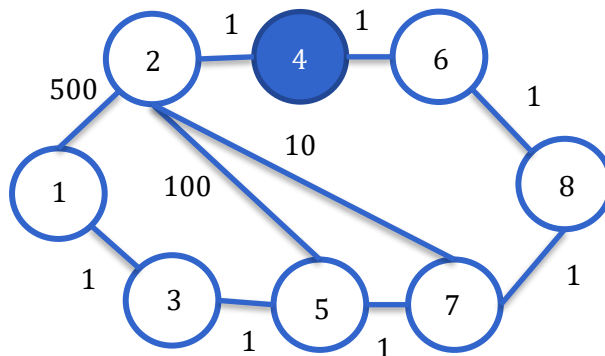


| vertices | 1 | 2   | 3 | 4   | 5   | 6         | 7   | 8         |
|----------|---|-----|---|-----|-----|-----------|-----|-----------|
| distance | 0 | 501 | 1 | 501 | 600 | $+\infty$ | 510 | $+\infty$ |

Queue

{3,1} {4,501} {5,600} {7,510}

■ نحذف العنصر في بداية queue ، نقارن وزنه مع أفضل وزن للعقدة نجدهم متساويين، فنفحص جيرانها ونضيفهم إلى queue ، ونحدث قيمهم في مصفوفة ال distance ، تذكر نضيف فقط العقد التي الطريق الجديد لها أصغر من أفضل طريق سابقا.

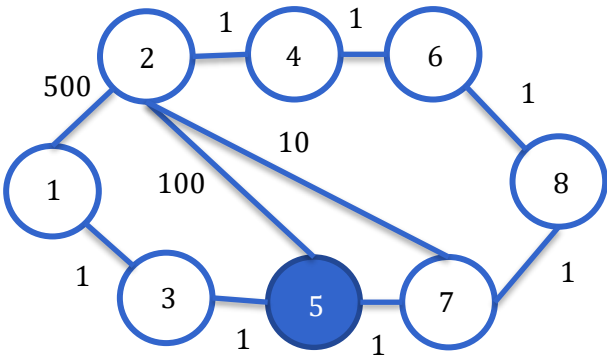


| vertices | 1 | 2   | 3 | 4   | 5 | 6         | 7   | 8         |
|----------|---|-----|---|-----|---|-----------|-----|-----------|
| distance | 0 | 500 | 1 | 501 | 2 | $+\infty$ | 510 | $+\infty$ |

Queue

{4,501} {5,600} {7,510} {5,2}

■ نحذف العنصر في بداية الـ queue، نقارن وزنه مع أفضل وزن للعقدة نجدهم متساويين، فنفحص جيرانها ونضيفهم إلى الـ queue، ونحدث قيمهم في مصفوفة الـ distance.

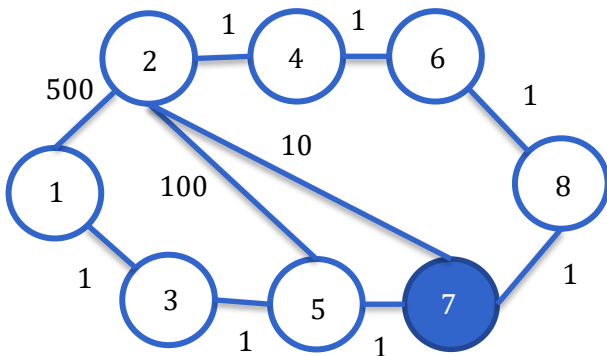


| vertices | 1 | 2   | 3 | 4   | 5 | 6   | 7   | 8         |
|----------|---|-----|---|-----|---|-----|-----|-----------|
| distance | 0 | 500 | 1 | 501 | 2 | 502 | 510 | $+\infty$ |

Queue

|         |         |       |         |
|---------|---------|-------|---------|
| {5,600} | {7,510} | {5,2} | {6,502} |
|---------|---------|-------|---------|

■ نحذف العنصر في بداية الـ queue، نقارن وزنه مع أفضل وزن للعقدة نجده أكبر، لذلك نتجاهله ونتابع.

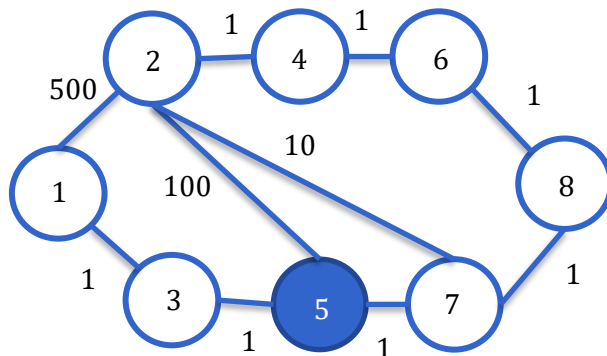


| vertices | 1 | 2   | 3 | 4   | 5 | 6   | 7   | 8         |
|----------|---|-----|---|-----|---|-----|-----|-----------|
| distance | 0 | 500 | 1 | 501 | 2 | 502 | 510 | $+\infty$ |

Queue

|         |       |         |
|---------|-------|---------|
| {7,510} | {5,2} | {6,502} |
|---------|-------|---------|

■ نحذف العنصر في بداية الـ queue، نقارن وزنه مع أفضل وزن للعقدة نجدهم متساويين، فنفحص جيرانها ونضيفهم إلى الـ queue، ونحدث قيمهم في مصفوفة الـ distance.

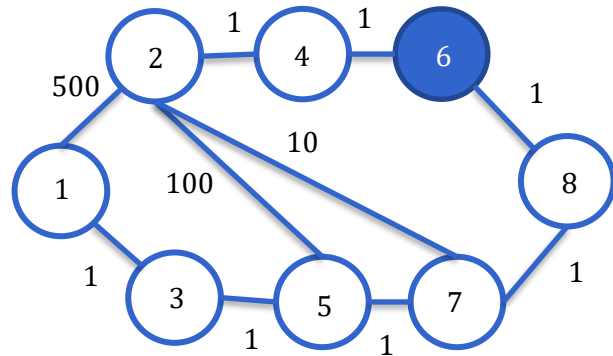


| vertices | 1 | 2   | 3 | 4   | 5 | 6   | 7   | 8   |
|----------|---|-----|---|-----|---|-----|-----|-----|
| distance | 0 | 500 | 1 | 501 | 2 | 502 | 510 | 511 |

Queue

|       |         |         |
|-------|---------|---------|
| {5,2} | {6,502} | {8,511} |
|-------|---------|---------|

■ نحذف العنصر في بداية الـ queue ، نقارن وزنه مع أفضل وزن للعقدة نجدهم متساويين ، فنفحص جيرانها ونضيفهم إلى الـ queue ، ونحدث قيمهم في مصفوفة الـ distance

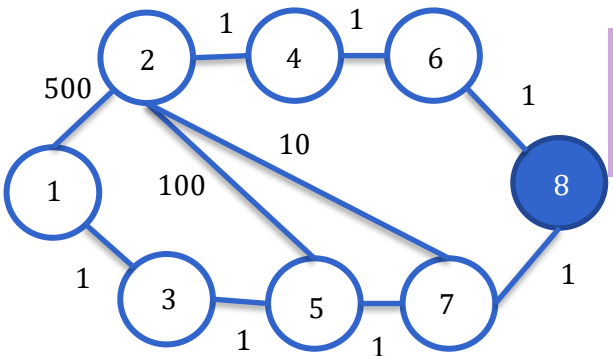


| vertices | 1 | 2   | 3 | 4   | 5 | 6   | 7 | 8   |
|----------|---|-----|---|-----|---|-----|---|-----|
| distance | 0 | 102 | 1 | 501 | 2 | 502 | 3 | 511 |

Queue

|         |         |         |       |
|---------|---------|---------|-------|
| {6,502} | {8,511} | {2,102} | {7,3} |
|---------|---------|---------|-------|

■ نحذف العنصر في بداية الـ queue ، نقارن وزنه مع أفضل وزن للعقدة نجدهم متساويين ، فنفحص جيرانها ونضيفهم إلى الـ queue ، ونحدث قيمهم في مصفوفة الـ distance.

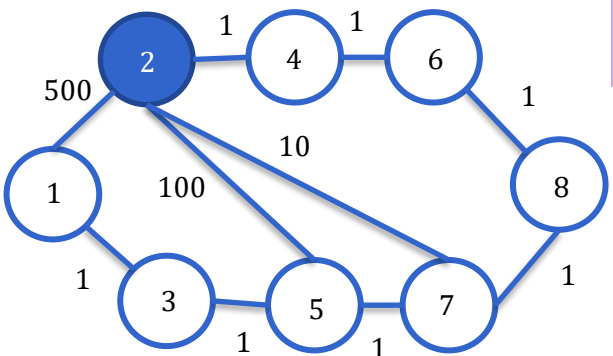


| vertices | 1 | 2   | 3 | 4   | 5 | 6   | 7 | 8   |
|----------|---|-----|---|-----|---|-----|---|-----|
| distance | 0 | 102 | 1 | 501 | 2 | 502 | 3 | 503 |

Queue

|         |         |       |         |
|---------|---------|-------|---------|
| {8,511} | {2,102} | {7,3} | {8,503} |
|---------|---------|-------|---------|

■ نحذف العنصر في بداية الـ queue ، نقارن وزنه مع أفضل وزن للعقدة نجده أكبر ، لذلك نتجاهله ونتابع.

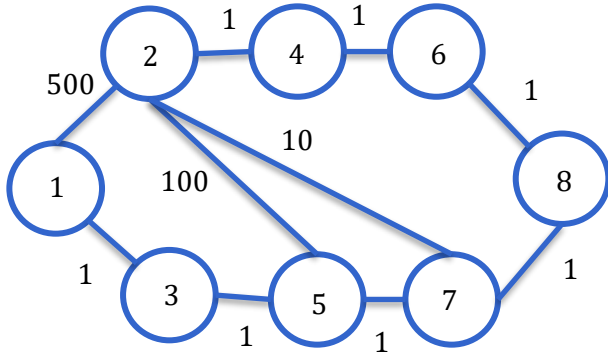


| vertices | 1 | 2   | 3 | 4   | 5 | 6   | 7 | 8   |
|----------|---|-----|---|-----|---|-----|---|-----|
| distance | 0 | 102 | 1 | 501 | 2 | 502 | 3 | 503 |

Queue

|         |       |         |
|---------|-------|---------|
| {2,102} | {7,3} | {8,503} |
|---------|-------|---------|

نحذف العنصر في بداية الـ queue ، نقارن وزنه مع أفضل وزن للعقدة نجدهم متساويين، فنفحص جيرانها ونضيفهم إلى الـ queue ، ونحدث قيمهم في مصفوفة الـ distance. لاحظ أننا سنعيد المرور على عقد مررنا عليها سابقاً ونحدث قيمها، ولاحقاً عندما نصل إلى {7,3} ستتحدث قيمة العقدة 2 لتصبح 13، وسنعيد المرور ثانية على عقد مررنا عليها من قبل، وكلما حصلنا على وزن أصغر سنكرر الأمر، سنكرر الخطوات السابقة للعديد من المرات حتى نحصل على الجواب النهائي وهو عندما يصبح الـ queue فارغاً.



| vertices | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|----------|---|---|---|---|---|---|---|---|
| distance | 0 | 7 | 1 | 6 | 2 | 5 | 3 | 4 |

ملاحظة:

قمنا باستخدام PriorityQueue بدلاً من Queue لتسريع العملية

## Code

```
import java.util.*;

class Node implements Comparator<Node> {
    public int node;
    public int cost;
    public Node() {}
    public Node(int node, int cost) {
        this.node = node;
        this.cost = cost;
    }
    @Override
    public int compare(Node a, Node b) {
        return Integer.compare(a.cost, b.cost);
    }
}
```

## شرح الخوارزمية :

■ (1-5) : تعريف صف Node الذي يمثل كل عقدة في الرسم البياني مع وزن كل منها.

■ (6-11) : تعريف دالة المقارنة داخل Node لترتيب العناصر في PriorityQueue حسب الكلفة (الأقل أولاً).



```
public class DijkstraPQ {
    private int[] dist;
    // Array to store shortest distances
    private Set<Integer> settled;
    // Set to keep track of visited nodes
    private PriorityQueue<Node> pq;
    // Priority queue for efficient min-cost retrieval
    private int V;
    // Number of vertices
    private List<List<Node>> adj;
    // Adjacency list

    public DijkstraPQ(int V) {
        this.V = V;
        dist = new int[V];
        settled = new HashSet<>();
        pq = new PriorityQueue<>(V, new Node());
    }

    public void dijkstra(List<List<Node>> adj, int src) {
        this.adj = adj;
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;
        pq.add(new Node(src, 0));
        while (settled.size() != V) {
            if (pq.isEmpty()) return;
            int u = pq.remove().node;
            if (settled.contains(u)) continue;
            settled.add(u);
            processNeighbors(u);
        }
    }
}
```

■ (13-20) : تعريف الصف الرئيسي

DijkstraPQ وتهيئة المتغيرات الرئيسية:

عدد العقد، مصفوفة المسافات، مجموعة

العقد المعالجة، وطابور الأولوية.

■ (22-28) : المنشئ

DijkstraPQ(int V) يقوم بتهيئة جميع

الهياكل الأساسية للخوارزمية

■ (30-38) : دالة dijkstra تبدأ بتهيئة

المسافات لكل العقد وتحديد كلفة المصدر

بصفر، ثم تضيفه إلى الطابور.

■ (40-47) : تنفيذ الخوارزمية: يتم سحب

العقدة ذات الكلفة الأقل من pq، وإذا لم

تعالج بعد تُضاف إلى مجموعة settled.



```
private void processNeighbors(int u) {
    for (Node neighbor : adj.get(u)) {
        int v = neighbor.node;
        int weight = neighbor.cost;
        if (!settled.contains(v)) {
            int newDist = dist[u] + weight;
            if (newDist < dist[v]) {
                dist[v] = newDist;
                pq.add(new Node(v, newDist));
            }
        }
    }
}
```

- (48-58): دالة processNeighbors تمرّ على كل جيران العقدة الحالية، وتحسب كلفة الوصول إليهم، وتحدث المسافة إن وُجد طريق أقصر، وتضيفهم إلى الطابور.

```
public static void main(String[] args) {
    int V = 5;
    int src = 0;
    List<List<Node>> adj = new ArrayList<>();
    for (int i = 0; i < V; i++) {
        adj.add(new ArrayList<>());
    }
    // Add edges (from → to, cost)
    adj.get(0).add(new Node(1, 9));
    adj.get(0).add(new Node(2, 6));
    adj.get(0).add(new Node(3, 5));
    adj.get(0).add(new Node(4, 3));
    adj.get(2).add(new Node(1, 2));
    adj.get(2).add(new Node(3, 4));
    DijkstraPQ graph = new DijkstraPQ(V);
    graph.dijkstra(adj, src);
    System.out.println("Shortest distances from node " + src + ":");
    for (int i = 0; i < V; i++) {
        System.out.println("To node" + i + " = " + (graph.dist[i] == Integer.MAX_VALUE ? "∞" : graph.dist[i]));
    }
}
```



- (60-66) : في main، يتم تحديد عدد العقد، وإنشاء قائمة المجاورات (البيان).
- (68-73) : إضافة الحواف (العلاقات بين العقد) إلى البيان (مثال تطبيقي).
- (75-76) : تشغيل الخوارزمية من المصدر.
- (78-81) : طباعة النتائج: المسافات الأقصر من المصدر إلى باقي العقد

# THE END